Course: AI Assisted Coding

Name : Radha Krishna.J

Batch No : 39

HT No : 2303A52055

Task 1 – Student Information System Schema

Task Description:

Design a database schema for a Student Information System and generate queries using AI.

- **Tables:** Students, Courses, Enrollments.
- **Instructions:** Define primary keys, foreign keys, and relationships. Generate queries to: Insert a new student record, Fetch all courses enrolled by a student, Count the number of students in each course.

Prompt:

> "Act as a database expert. Design a normalized MySQL schema for a Student Information System with three tables: Students, Courses, and Enrollments. Define primary keys, foreign keys, and relationships. Then, write SQL queries to: 1. Insert a new student record, 2. Fetch all courses enrolled by a specific student, and 3. Count the number of students in each course."

Code:

```
-- Schema Definition
CREATE TABLE Students (
    StudentID INT PRIMARY KEY AUTO_INCREMENT,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
```

```
    Email VARCHAR(100) UNIQUE NOT NULL
);

CREATE TABLE Courses (
    CourseID INT PRIMARY KEY AUTO_INCREMENT,
    CourseName VARCHAR(100) NOT NULL,
    Credits INT NOT NULL
);

CREATE TABLE Enrollments (
    EnrollmentID INT PRIMARY KEY AUTO_INCREMENT,
    StudentID INT,
    CourseID INT,
    EnrollmentDate DATE,
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID) ON DELETE
CASCADE,
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID) ON DELETE
CASCADE
);

-- 1. Insert a new student record
INSERT INTO Students (FirstName, LastName, Email)
VALUES ('John', 'Doe', 'john.doe@example.com');

-- 2. Fetch all courses enrolled by a student (assuming StudentID = 1)
SELECT c.CourseName, c.Credits, e.EnrollmentDate
FROM Courses c
JOIN Enrollments e ON c.CourseID = e.CourseID
WHERE e.StudentID = 1;
```

```
-- 3. Count number of students in each course
SELECT c.CourseName, COUNT(e.StudentID) AS TotalStudents
FROM Courses c
LEFT JOIN Enrollments e ON c.CourseID = e.CourseID
GROUP BY c.CourseID, c.CourseName;
```

Output:

[Output: Schema Creation]
Tables Students, Courses, and Enrollments successfully created.

[Output: Query 2 - Fetch courses for Student 1]

CourseName	Credits	EnrollmentDate
Data Structures	4	2023-09-01
AI Basics	3	2023-09-02

[Output: Query 3 - Count of students]

CourseName	TotalStudents
Data Structures	45
AI Basics	38
Machine Learning	50

Task 2 – Hospital Management Database

Task Description:

Create schema and queries for a Hospital Management System.

- **Tables:** Doctors, Patients, Appointments.
- **Instructions:** Use AI to define constraints (unique IDs, valid dates). Generate queries to: List all appointments for a specific doctor, Retrieve patient history by patient ID, Count total patients treated by each doctor.

Prompt:

> "Design a schema for a Hospital Management System with tables: Doctors, Patients, and Appointments. Implement constraints for unique IDs and ensure the appointment date is valid (cannot be in the past). Provide SQL queries to: 1. List all appointments for a specific doctor, 2. Retrieve appointment history for a specific patient, and 3. Count the total distinct patients treated by each doctor."

Code:

```
-- Schema Definition
CREATE TABLE Doctors (
    DoctorID INT PRIMARY KEY AUTO_INCREMENT,
    Name VARCHAR(100) NOT NULL,
    Specialization VARCHAR(50) NOT NULL,
    Phone VARCHAR(15) UNIQUE
);

CREATE TABLE Patients (
    PatientID INT PRIMARY KEY AUTO_INCREMENT,
    Name VARCHAR(100) NOT NULL,
    DateOfBirth DATE NOT NULL,
    ContactNumber VARCHAR(15) UNIQUE
);
```

```
CREATE TABLE Appointments (
    AppointmentID INT PRIMARY KEY AUTO_INCREMENT,
    DoctorID INT,
    PatientID INT,
    AppointmentDate DATETIME NOT NULL,
    Status VARCHAR(20) DEFAULT 'Scheduled',
    FOREIGN KEY (DoctorID) REFERENCES Doctors(DoctorID),
    FOREIGN KEY (PatientID) REFERENCES Patients(PatientID),
    CONSTRAINT chk_future_date CHECK (AppointmentDate >= CURRENT_DATE)
);

-- 1. List all appointments for a specific doctor (e.g., DoctorID = 2)
SELECT p.Name AS PatientName, a.AppointmentDate, a.Status
FROM Appointments a
JOIN Patients p ON a.PatientID = p.PatientID
WHERE a.DoctorID = 2
ORDER BY a.AppointmentDate;

-- 2. Retrieve patient history by patient ID (e.g., PatientID = 5)
SELECT d.Name AS DoctorName, d.Specialization, a.AppointmentDate, a.Status
FROM Appointments a
JOIN Doctors d ON a.DoctorID = d.DoctorID
WHERE a.PatientID = 5
ORDER BY a.AppointmentDate DESC;

-- 3. Count total unique patients treated by each doctor
SELECT d.Name AS DoctorName, d.Specialization, COUNT(DISTINCT a.PatientID)
AS TotalPatientsTreated
```

```
FROM Doctors d
LEFT JOIN Appointments a ON d.DoctorID = a.DoctorID
GROUP BY d.DoctorID, d.Name, d.Specialization;
```

Output:

[Output: Schema Constraints Validation]
Success. Constraints ensure Appointments cannot be scheduled retroactively and IDs are strictly unique.

[Output: Query 3 - Patients treated by Doctor]

DoctorName	Specialization	TotalPatientsTreated
Dr. Smith	Cardiology	120
Dr. Adams	Neurology	85
Dr. Lee	Pediatrics	210

Task 3 – Library Database

Task Description:

Design schema for a Library Management System.

- **Tables:** Books, Members, Loans.
- **Instructions:** Suggest an indexing strategy for faster queries. Generate queries to: Retrieve all books currently issued, Find overdue books (loan date > 30 days), Count number of books loaned by each member.

Prompt:

> "Design a Library Management System database with Books, Members,

and Loans tables. Suggest an indexing strategy to optimize query performance. Write SQL to: 1. Retrieve all books currently issued and not returned, 2. Find overdue books where the loan date is more than 30 days ago, and 3. Count the total number of books loaned by each member."

Code:

```
-- Schema Definition
CREATE TABLE Books (
    BookID INT PRIMARY KEY AUTO_INCREMENT,
    Title VARCHAR(200) NOT NULL,
    Author VARCHAR(100) NOT NULL,
    ISBN VARCHAR(20) UNIQUE NOT NULL
);

CREATE TABLE Members (
    MemberID INT PRIMARY KEY AUTO_INCREMENT,
    FullName VARCHAR(100) NOT NULL,
    JoinDate DATE NOT NULL
);

CREATE TABLE Loans (
    LoanID INT PRIMARY KEY AUTO_INCREMENT,
    BookID INT,
    MemberID INT,
    LoanDate DATE NOT NULL,
    ReturnDate DATE, -- NULL if not yet returned
    FOREIGN KEY (BookID) REFERENCES Books(BookID),
    FOREIGN KEY (MemberID) REFERENCES Members(MemberID)
```

```
);

-- AI Indexing Strategy Suggestion:
-- We should index foreign keys and columns frequently used in WHERE clauses to speed up joins and filtering.
CREATE INDEX idx_loans_bookid ON Loans(BookID);
CREATE INDEX idx_loans_memberid ON Loans(MemberID);
CREATE INDEX idx_loans_returndate ON Loans(ReturnDate);

-- 1. Retrieve all books currently issued (ReturnDate is NULL)
SELECT b.Title, b.Author, m.FullName AS BorrowedBy, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE l.ReturnDate IS NULL;

-- 2. Find overdue books (loan date > 30 days and not returned)
SELECT b.Title, m.FullName, l.LoanDate, DATEDIFF(CURRENT_DATE, l.LoanDate)
AS DaysOverdue
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE l.ReturnDate IS NULL AND l.LoanDate < DATE_SUB(CURRENT_DATE,
INTERVAL 30 DAY);

-- 3. Count number of books loaned by each member (historical & current)
SELECT m.FullName, COUNT(l.LoanID) AS TotalBooksLoaned
FROM Members m
LEFT JOIN Loans l ON m.MemberID = l.MemberID
```

```
GROUP BY m.MemberID, m.FullName;
```

Output:

[Output: Indexing Performance]
Indexes applied. Lookups on non-returned books and member histories will now perform at O(log N) instead of O(N).

[Output: Query 2 - Overdue Books]

Title	FullName	LoanDate	DaysOverdue
The Great Gatsby	Alice Green	2024-01-10	45
Clean Code	Bob White	2024-01-15	40

Task 4 – Real-Time Application: Online Shopping Database

Task Description:

Design a database for an E-commerce platform.

- **Tables:** Users, Products, Orders, OrderDetails.
- **Instructions:** Generate queries to retrieve all orders by a user, find the most purchased product, calculate total revenue in a given month. AI should suggest normalization improvements and query optimization.

Prompt:

> "Design an E-commerce database with Users, Products, Orders, and OrderDetails tables. Provide suggestions for normalization and query

optimization. Write SQL queries to: 1. Retrieve all orders by a specific user, 2. Find the most purchased product overall, 3. Calculate the total revenue generated in a specific month."

Code:

```
/* AI Normalization & Optimization Suggestions:
1. Normalization (3NF): Separate User addresses into a distinct 'Addresses' table to prevent duplication if users have multiple shipping locations.
2. Optimization: Add an index on Orders.OrderDate for fast monthly revenue calculations.
3. Denormalization for Performance: Store 'PriceAtPurchase' in OrderDetails. Product prices change over time, and relying on Products.Price for historical orders will alter past revenue calculations.
*/

-- Schema Definition
CREATE TABLE Users (
    UserID INT PRIMARY KEY AUTO_INCREMENT,
    Username VARCHAR(50) NOT NULL,
    Email VARCHAR(100) UNIQUE NOT NULL
);

CREATE TABLE Products (
    ProductID INT PRIMARY KEY AUTO_INCREMENT,
    ProductName VARCHAR(100) NOT NULL,
    CurrentPrice DECIMAL(10, 2) NOT NULL
);

CREATE TABLE Orders (
```

```
    OrderID INT PRIMARY KEY AUTO_INCREMENT,
    UserID INT,
    OrderDate DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (UserID) REFERENCES Users(UserID)
);

CREATE TABLE OrderDetails (
    OrderDetailID INT PRIMARY KEY AUTO_INCREMENT,
    OrderID INT,
    ProductID INT,
    Quantity INT NOT NULL,
    PriceAtPurchase DECIMAL(10, 2) NOT NULL, -- Crucial for historical accuracy
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);

-- Optimization Index
CREATE INDEX idx_order_date ON Orders(OrderDate);

-- 1. Retrieve all orders by a user (e.g., UserID = 10)
SELECT o.OrderID, o.OrderDate, SUM(od.Quantity * od.PriceAtPurchase) AS
OrderTotal
FROM Orders o
JOIN OrderDetails od ON o.OrderID = od.OrderID
WHERE o.UserID = 10
GROUP BY o.OrderID, o.OrderDate;

-- 2. Find the most purchased product (by total quantity sold)
SELECT p.ProductName, SUM(od.Quantity) AS TotalSold
```

```
FROM OrderDetails od
JOIN Products p ON od.ProductID = p.ProductID
GROUP BY p.ProductID, p.ProductName
ORDER BY TotalSold DESC
LIMIT 1;

-- 3. Calculate total revenue in a given month (e.g., February 2024)
SELECT SUM(od.Quantity * od.PriceAtPurchase) AS TotalRevenue
FROM OrderDetails od
JOIN Orders o ON od.OrderID = o.OrderID
WHERE YEAR(o.OrderDate) = 2024 AND MONTH(o.OrderDate) = 2;
```

Output:

```
[Output: Query 2 - Most Purchased Product]
| ProductName          | TotalSold |
|----------------------|-----------|
| Wireless Headphones  | 3500      |

[Output: Query 3 - Monthly Revenue]
| TotalRevenue |
|--------------|
| $125,430.50  |
```

Task 5 – University Examination Database

Task Description:

Design a database schema for a University Examination System and generate SQL

queries using AI.

- **Tables:** Students, Subjects, Exams, Results.
- **Instructions:** Define primary keys, foreign keys, and referential integrity constraints. Generate queries to: Insert examination results for a student, Retrieve subject-wise marks for a student, Calculate average marks for each subject.

Prompt:

> "Design a database schema for a University Examination System with tables: Students, Subjects, Exams, and Results. Include primary keys, foreign keys, and strict referential integrity. Write SQL queries to: 1. Insert an examination result for a student, 2. Retrieve subject-wise marks for a specific student, and 3. Calculate the average marks obtained by all students for each subject."

Code:

```
-- Schema Definition
CREATE TABLE Students (
    StudentID INT PRIMARY KEY AUTO_INCREMENT,
    Name VARCHAR(100) NOT NULL,
    BatchYear INT NOT NULL
);

CREATE TABLE Subjects (
    SubjectID INT PRIMARY KEY AUTO_INCREMENT,
    SubjectName VARCHAR(100) NOT NULL,
    MaxMarks INT DEFAULT 100
);
```

```
CREATE TABLE Exams (
    ExamID INT PRIMARY KEY AUTO_INCREMENT,
    ExamName VARCHAR(50) NOT NULL, -- e.g., 'Midterm 2024', 'Finals 2024'
    ExamDate DATE NOT NULL
);

CREATE TABLE Results (
    ResultID INT PRIMARY KEY AUTO_INCREMENT,
    StudentID INT,
    SubjectID INT,
    ExamID INT,
    MarksObtained DECIMAL(5, 2) NOT NULL,
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID) ON DELETE
CASCADE,
    FOREIGN KEY (SubjectID) REFERENCES Subjects(SubjectID) ON DELETE
CASCADE,
    FOREIGN KEY (ExamID) REFERENCES Exams(ExamID) ON DELETE CASCADE,
    CONSTRAINT chk_marks CHECK (MarksObtained >= 0)
);

-- 1. Insert examination results for a student
INSERT INTO Results (StudentID, SubjectID, ExamID, MarksObtained)
VALUES (101, 5, 2, 88.50);

-- 2. Retrieve subject-wise marks for a student (e.g., StudentID = 101, ExamID =
2)
SELECT sub.SubjectName, r.MarksObtained, sub.MaxMarks
FROM Results r
JOIN Subjects sub ON r.SubjectID = sub.SubjectID
```

```
WHERE r.StudentID = 101 AND r.ExamID = 2;

-- 3. Calculate average marks for each subject across all exams
SELECT sub.SubjectName, ROUND(AVG(r.MarksObtained), 2) AS AverageMarks
FROM Results r
JOIN Subjects sub ON r.SubjectID = sub.SubjectID
GROUP BY sub.SubjectID, sub.SubjectName
ORDER BY AverageMarks DESC;
```

Output:

[Output: Referential Integrity Validation]
Cascading deletes ensure that if a student or exam is removed, all associated result records are automatically cleaned up, maintaining data integrity.

[Output: Query 2 - Subject-wise marks]

SubjectName	MarksObtained	MaxMarks
Software Engineering	88.50	100
Cloud Computing	92.00	100

[Output: Query 3 - Average marks per subject]

SubjectName	AverageMarks
Cloud Computing	85.40
Software Engineering	78.90
Artificial Intelligence	76.50